

RAG Project Experiments Report

All experiments were run on the project's own resume document, using the same embedding pipeline (sentence-transformers) and LLM (Claude via OpenRouter) as the deployed application.

Experiment 1: Chunk Size

Objective: measure how chunk size (300, 500, 1000 words, fixed 50-word overlap) affects the number of chunks generated, embedding time, and retrieval relevance (top cosine similarity score averaged across three test questions).

Chunk Size	# Chunks	Avg Words/Chunk	Embed Time (s)	Avg Top Score
300	3	200.7	0.37	0.303
500	2	288.0	0.24	0.311
1000	1	526.0	0.08	0.299

Screenshot Experiment

```
PS C:\Users\DELL\Desktop\Internship_Notes\wk2 n\RAG_Website\experiments> python exp1_chunk_size.py
Chunk Size #Chunks Avg Words Embed Time(s) Top Score (avg)
300        3      200.7    0.37      0.303
500        2      288.0    0.24      0.311
1000       1      526.0    0.08      0.299
```

Discussion of Impact

On this single-page resume document, chunk size had only a marginal effect on retrieval score (0.299–0.311 across all sizes), because the source document is short enough that even a 1000-word chunk captures nearly the entire document in a single unit at chunk size 1000, only one chunk was produced. The clearest measurable effect was on chunk count and embedding time: smaller chunks (300 words) produced 3 chunks and took longer to embed (0.37s) than the single 1000-word chunk (0.08s), since more separate pieces of text must be individually encoded.

For longer, multi-page documents, this experiment would be expected to show a more pronounced trade-off: very large chunks (1000+ words) dilute relevance because multiple unrelated topics get merged into one vector, lowering precision, while very small chunks (300 words) risk splitting a coherent idea across two chunks and losing surrounding context. A mid-range chunk size (500 words) is a reasonable default that balances these two failure modes, which matches the value already used as the default in this project's implementation.

Experiment 2: Chunk Overlap

Objective: with chunk size fixed at 500 words, compare overlap values of 0, 50, 100, and 150 words and measure their effect on retrieval score.

Overlap	# Chunks	Avg Top Score
0	2	0.303
50	2	0.311
100	2	0.299
150	2	0.299

Screenshot

```

PS C:\Users\DELL\Desktop\Internship_Notes\wk2 n\RAG_Website\experiments> python exp2_overlap.py
Overlap  #Chunks  Avg Top Score
0        2        0.303
50       2        0.311
100      2        0.299
150      2        0.299

```

Discussion of Impact

Overlap had a small but real effect: a 50-word overlap produced the highest average retrieval score (0.311), slightly outperforming zero overlap (0.303) and larger overlaps of 100–150 words (0.299 each). This suggests a modest overlap helps preserve context at chunk boundaries without meaningfully increasing noise. Overlap beyond a certain point stopped improving results here, likely because on a short document, larger overlaps begin duplicating a large fraction of the same content across chunks without adding new context, while also increasing the number of near-duplicate vectors the retriever has to sift through. A 50–80 word overlap is a reasonable default for typical document lengths.

Experiment 3: Prompt Templates

Objective: compare three prompt construction strategies a minimal prompt, a strictly grounded prompt with an explicit fallback instruction, and a role-based prompt asking the model to note which context it used using the same retrieved context and question for all three.

Basic prompt

Produced a competent, well-structured answer, but with no explicit instruction to stay grounded in the context or acknowledge missing information, leaving more room for the model to add unsupported detail on harder questions.

```

PS C:\Users\DELL\Desktop\Internship_Notes\wk2 n\RAG_Website\experiments> python exp3_prompts.py

=== Basic ===
# Projects Built by Ali Haider

## 1. **Eye-Controlled Mouse System**
A hands-free virtual mouse that controls cursor movement and click actions through eye movement and blink detection.

**Tools Used:**
- Python, OpenCV, MediaPipe, PyAutoGUI, NumPy

**Key Features:**
- Real-time facial landmark tracking (468 points via MediaPipe Face Mesh)
- Eye position mapping to screen coordinates
- Blink detection for click triggers
- ~20-25 FPS performance on standard laptop webcam

```

Strict grounding prompt

Produced a more conservative, source-anchored answer and explicitly enumerated projects and tools drawn directly from the context. This style is the most defensible for an enterprise RAG system, since it explicitly instructs the model to acknowledge when information is not present rather than guessing.

```

=== Strict grounding ===
Based on the context, this person has built the following projects:

1. **Eye-Controlled Mouse System**
- Tools: Python, OpenCV, MediaPie, PyAutoGUI, NumPy
- A hands-free virtual mouse that controls cursor movement and clicks through eye movement and blink detection

2. **Face Mask Detection System**
- Tools: Python, Ultralytics YOLOv8, Roboflow, OpenCV, Google Colab
- A YOLOv8 object detection model trained to classify faces into three categories: with mask, without mask, and incorrectly worn mask

3. **Automated Question Generation System**
- Tools: Python, Flask, spaCy, NLTK, HTML5, CSS3
- An NLP pipeline that generates relevant comprehension questions from text input, integrated into a Flask web application with a browser

```

Role + citation prompt

Produced output very similar in quality to the strict grounding version, with added structure (bolded project names, specific metrics like “85% mAP@0.5” preserved accurately from the source). Framing the model as a precise research assistant appeared to slightly improve formatting consistency without sacrificing grounding.

```

=== Role + citation ===
# Projects and Tools Used

Based on the context, Ali Haider has built three main projects:

## 1. Eye-Controlled Mouse System
**Tools:** Python, OpenCV, MediaPie, PyAutoGUI, NumPy
- Built a hands-free virtual mouse system controlled through eye movement and blink detection

## 2. Face Mask Detection System
**Tools:** Python, Ultralytics YOLOv8, Roboflow, OpenCV, Google Colab
- Trained a YOLOv8 object detection model to classify faces into three mask-wearing categories
- Achieved approximately 85% mAP@0.5 on validation set

```

Discussion of Impact

All three templates produced factually correct answers on this test question, since the retrieved context was highly relevant. The practical difference between prompt styles becomes more significant on ambiguous questions or when retrieved context is weak — the strict grounding and role-based prompts are expected to more reliably decline or hedge when the answer is not actually present in the context, whereas the basic prompt gives the model more latitude to fill gaps from its own training knowledge, increasing hallucination risk. The project's deployed prompt (`services/llm_service.py`) follows the strict-grounding pattern for this reason.

Experiment 4: Embedding Models

Objective: compare the project's default embedding model (all-MiniLM-L6-v2) against a larger alternative (all-mpnet-base-v2) on embedding speed and retrieval relevance.

Model	Embed Time (s)	Avg Top Score
all-MiniLM-L6-v2	0.19	0.311
all-mpnet-base-v2	1.96	0.334

Discussion of Impact

all-mpnet-base-v2 produced a higher average retrieval score (0.334 vs. 0.311), consistent with its larger architecture (420MB vs. ~80MB) and its known stronger performance on semantic similarity benchmarks. However, it was roughly 10x slower to embed the same chunks (1.96s vs. 0.19s) and requires substantially more disk space and memory. For a small personal knowledge base like this project's, all-MiniLM-L6-v2 remains the more practical choice, since the speed advantage matters more than the modest relevance gain at this scale. For an enterprise deployment indexing large document sets where retrieval precision has a bigger impact on business outcomes, the accuracy gain from a larger model such as all-mpnet-base-v2 (or a proprietary API-based embedding model) may justify the added latency and cost.

```
PS C:\Users\DELL\Desktop\Internship_Notes\wk2 n\RAG_Website\experiments> python ex4_embedding_models.py
Model Embed Time(s) Avg Top Score
'ReadTimeoutError("HTTPConnectionPool(host='huggingface.co', port=443): Read timed out. (read timeout=10)", '(Request ID : 9f0b1c12-52bd-4b79-aa7a-e5706cd0de09))' thrown while requesting HEAD https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2/resolve/main/tokenizer_config.json
Retrying in 1s [Retry 1/5].
all-MiniLM-L6-v2 0.19 0.311
modules.json: 100%|██████████████████████████████████████████████████████████████████████████████| 349/349 [00:00<00:00, 813kB/s]
C:\Users\DELL\AppData\Roaming\Python\Python314\site-packages\huggingface_hub\file_download.py:143: UserWarning: `huggingface_hub` cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support them in C:\Users\DELL\.cache\huggingface\hub\models--sentence-transformers--all-mpnet-base-v2. Caching files will still work but in a degraded version that might require more space on your disk. This warning can be disabled by setting the `HF_HUB_DISABLE_SYMLINKS_WARNING` environment variable. For more details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
```

Overall Findings

- Chunk size and overlap had a measurable but modest effect on retrieval score for a short document; the effect is expected to be larger on longer, multi-topic documents.
- A 500-word chunk size with 50-word overlap is a reasonable default, balancing chunk count, embedding cost, and retrieval quality.
- Prompt design did not change factual correctness on a well-matched question, but strict grounding instructions are expected to reduce hallucination on harder or ambiguous questions, and were adopted as the project's default.
- Larger embedding models can meaningfully improve retrieval relevance, but at a significant cost in embedding speed; model choice should be driven by the scale and latency requirements of the deployment, not accuracy alone.